

Git and GitHub

Version control System (VCS):

Tool or software that helps you to track changes made to files over time. It lets you:

- Save different versions of projects
- Go back to an earlier version
- Work with team
- See who made change what and when

Examples of Version Control System (VCS)?

Examples of VCS:

NAME	TYPE	DESCRIPTION
Git	Distributed VCS	Most popular. Used by developers to track code changes.
Subversion (SVN)	Centralized VCS	Older system, all files stored in a central server.
Mercurial	Distributed VCS	Similar to Git, used for performance and simplicity.
CVS	Centralized VCS	One of the first version control systems, now outdated.
Perforce	Centralized VCS	Used in large enterprises and gaming companies.

•**Centralized VCS** – All data is stored in one place (e.g., SVN).

•**Distributed VCS** – Every user has a full copy (e.g., Git, Mercurial).



GitHub What is a GitHub?

GitHub is a website where you can store your **Git Projects** online.

Launched by:

Tom Preston-Werner, Chris Wanstrath, and PJ Hyett in 2008.

Owned by:

Microsoft Since 2018.

- Online hosting of Git repositories (Folder) **Repo**
- Team collaboration and code sharing
- Issue tracking for bugs and tasks
- Public and private repositories



What is a Git?

It's a **version control system** used to track changes in code or files.

Created by: **Linux Torvalds** in 2005.

- Free and open-source
- Distributed version control system
- Fast, Secure and reliable
- Tracks changes with full history
- Enables team collaboration



What is a Git Bash?

A **command-line tool** (terminal) that allows you to run **Git commands**. Comes bundled when you install Git on Windows.

Things to do

Things To Do



1. Create GitHub Account



2. Download & Install Git/Git Bash



3. Download & Install Code Editor

Visual Studio Code (OR) [VS Code](#)

What is GitHub Repository:



What is Repository?

It is a **Digital Folder** for your project, It make easy to manage and share your work with others.

Inside this folder, you keep all your project's files: like code, images, documents, everything.

Also Called

RAPO



Git Add & Commit

In Git, **add** and **commit** are two basic steps to save changes in your project.

Step 01

add

add tells Git which files you want to include in your next save.

Step 02

commit

commit saves those changes with a message, so you have a record of what you did.

Git Configuration Commands

To configure git into your system or to link your local computer with github, you need to use following **git** commands.

```
git config --global user.name "github-username"
```

```
git config --global user.email "github-email"
```

```
git config --list
```

```
git config --global user.name "bilalramzan01"
```

We'll walk through: `git config --global user.name "Your Name"`

```
git config --global user.email "bilalramzan.cs@gmail.com"
```

```
git config --global --list
```

git clone

This command is used to copy a project (repository) from remote server (GitHub) to your computer.

It makes a full copy of the code and its history.

Syntax : `git clone <repository_url>`



Terminal Commands

cd (Change Directory)

This command is used to move from one folder (directory) to another in your computer through the terminal or command line.

Syntax : `cd <folder_name>`

Examples:

- ➡ `cd Documents` (To move inside of Documents Folder)
- ➡ `cd ..` (To go back or move to previous folder)
- ➡ `cd /home/user/Desktop`
(This directly takes you to the Desktop folder.)

Terminal Commands

ls (List)

- ◆ Shows a list of files and folders in the current directory (folder).

clear

- ◆ Clears the terminal screen to remove all previous commands.

pwd (Print Working Directory)

- ◆ Shows the full path of the current folder you are in.

git clone

Purpose: Download an existing repository from GitHub to your computer.

```
git clone https://github.com/user/repo.git
```

git status

Purpose: Check the current state of your repository.

```
git status
```

📌 Shows:

- modified files
- staged files
- untracked files

git add

Purpose: Add files to the staging area before committing.

```
git add file.txt
```

git commit

Purpose: Save changes permanently in Git.

```
git commit -m "Added login page"
```

📌 Always write a meaningful message.

git push

Purpose: Upload local commits to GitHub.

```
git push origin main
```

📌 Sends your code to the remote repository.

git pull

Purpose: Download the latest changes from GitHub.

`git pull origin main`

 Updates your local repository.

git branch

Purpose: Show or create branches.

Show branches:

`git branch`

Create new branch:

`git branch feature-login`

git checkout

Purpose: Switch between branches.

`git checkout feature-login`

git merge

Purpose: Combine branches.

`git merge feature-login`